

# Pokemon TCG Database Modeling Exercise:

Nicolas Casillas - A01787292

## Justification:

### 1. Player table

This table represents each user of the system. Central entity that connects gameplay, economy, and progression.

#### Attributes

Player\_id (PK) → uniquely identifies each player.

Username (VARCHAR) → public identifier; should be unique.

password (VARCHAR) → authentication (stored as hash in practice).

email (VARCHAR) → account recovery and communication.

level (INT) → progression system indicator.

creation\_date (DATE) → tracks account age and analytics.

This is the core actor. Almost every other table depends on the Player, which reflects real systems.

### 2. Card

This table represents all playable or collectible cards.

#### Attributes

card\_id (PK) → unique identifier.

name → card identity.

rarity → affects drop rates and value.

type → distinguishes Pokémon, Trainer, Energy.

health (INT, nullable) → only for Pokémon.

attack (INT, nullable) → only for Pokémon.

creation\_date → when added to the game.

image → visual representation (UI use).

Central to gameplay. Nullable stats are acceptable because not all cards share attributes.

### 3. Deck

This table represents a playable configuration of cards created by a player.

#### Attributes

Deck\_id (PK) → unique deck.

Player\_id (FK) → ownership.

name → user-defined identifier.

Separates ownership (Player) from configuration (Deck), enabling multiple decks per player.

#### 4. Deck\_Card (Bridge)

This table resolves the many-to-many relationship between Deck and Card.

##### Attributes

Deck\_card\_id (PK) → unique row.  
deck\_id (FK) → associated deck.  
card\_id (FK) → included card.  
Date\_added → tracking modifications.  
card\_quantity → number of copies in deck.

A deck contains many cards, and a card can appear in many decks → requires a bridge.  
card\_quantity avoids duplication of rows.

#### 5. Collection

This table represents a logical grouping of cards created by the player (not necessarily gameplay-valid).

##### Attributes

Collection\_id (PK)  
Player\_id (FK)  
name  
creation\_date

Improves UX by allowing players to organize cards (favorites, themes, etc.), independent of decks.

#### 6. Card\_in\_Collection (Bridge)

Many-to-many relationship between Collection and Card.

##### Attributes

Collection\_card\_id (PK)  
Collection\_id (FK)  
card\_id (FK)  
Date\_added  
card\_quantity

Same reasoning as Deck\_Card, but for organization instead of gameplay.

#### 7. Match

This table represents the game session between the two players.

##### Attributes

Match\_id (PK)  
Player1\_id (FK)  
Player2\_id (FK)  
winner (FK)  
update (DATETIME) → last state update

Explicitly models a 1v1 competitive interaction.  
Having both players as attributes enforces exactly two participants.

## **8. Match\_State**

This table stores the changes or state of a match over time.

### **Attributes**

match\_id (PK, FK)  
state\_id (PK) → distinguishes states  
Player\_in\_turn\_id (FK) → whose turn it is  
board (JSON/TEXT) → full board snapshot  
turn (INT) → turn number  
phase → game phase (draw, attack, etc.)

## **9. Store**

This table represents purchasable items in the game.

### **Attributes**

Item\_id (PK)  
name  
type → distinguishes packs, cosmetics, etc.  
price

Abstracts all purchasable elements into a single entity, simplifying the transaction system.

## **10. Transaction**

This table tracks purchases.

### **Attributes**

Transaction\_id (PK)  
Player\_id (FK)  
Item\_id (FK)  
Currency\_type → which currency was used  
date

## **11. Currency**

This table stores player balances.

### **Attributes**

Player\_id (FK)  
Crystals  
Coins  
Credits

Easier access and management of a player's economy

## 12. Ranked

This table represents competitive ranking.

### Attributes

Rank → player's position/tier

Player\_id (FK)

Season

Separates competitive progression from general player data.

Supports seasonal resets and ranking systems.

